

Sumários disponíveis no [ScienceDirect](#)

HardwareX

Página inicial da revista: www.elsevier.com/locate/ohx

Entrada/Saída distribuída como serviço: uma solução de aquisição de dados para a Indústria 4.0



Ricardo P. Pontarolli^y, Jeferson A. Bigheti, Felipe O. Domingues, Lucas BR de Sá, Eduardo P. Godoy

Instituto de Ciência e Tecnologia, Universidade Estadual Paulista (Unesp), Caixa Postal 18087-180, Sorocaba, SP, Brasil

informações do artigo

Histórico do artigo:

Recebido em 27 de outubro de 2021

Recebido em versão revisada em 16 de agosto de 2022.

Aceito em 27 de agosto de 2022

Palavras-chave:

E/S distribuída

Arquitetura orientada a serviços

Estrutura molecular

Automação industrial

resumo

A aquisição de dados é uma etapa essencial para qualquer aplicação industrial. A automação industrial tradicionalmente se baseia em Sistemas de Supervisão, Controle e Aquisição de Dados (SCADA), e a E/S remota é um elemento importante dessa infraestrutura. A E/S remota é responsável pela aquisição de dados de sensores e atuadores de processos industriais. A E/S distribuída representa uma evolução da E/S remota, incluindo processamento e suporte à comunicação em rede.

Com a adoção de tecnologias recentes na Indústria 4.0, as soluções de automação estão passando pela servitização e sendo desenvolvidas como um serviço. Este artigo descreve o desenvolvimento de uma solução de E/S distribuída, de código aberto e baixo custo, para automação industrial.

A solução baseia-se em um computador de placa única com módulos de expansão de E/S, uma estrutura de microsserviços e um software personalizado para permitir que a aquisição de dados de E/S funcione de forma padronizada, escalável e em rede. São apresentados alguns exemplos de aplicação do serviço de E/S distribuído, como redundância e histórico de E/S, controle de processos e monitoramento de IIoT (Internet Industrial das Coisas).

© 2022 Os Autores. Publicado pela Elsevier Ltd. Este é um artigo de acesso aberto sob a licença CC BY-NC-ND (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Hardware em contexto

Introdução

Um sistema de aquisição de dados (DAQ) é uma combinação de ferramentas utilizadas para coletar, analisar e registrar informações sobre algum fenômeno, como mostrado na [Figura 1](#). A aquisição de dados é a etapa inicial para qualquer tipo de aplicação científica. Para compreender um problema, é necessário coletar, processar e explorar as informações provenientes dos sinais obtidos do ambiente de interesse. Considerando o hardware de DAQ, existem três tipos gerais de uso para a aquisição de dados: em laboratório, portátil e remoto ou distribuído.

Os sistemas de aquisição de dados (DAQ) de laboratório e distribuídos geralmente consistem em hardware estático instalado em locais específicos e conectado a computadores de mesa de alguma forma. Esses sistemas dependem de software para acessar, processar, visualizar e analisar dados. A diferença entre DAQ de laboratório e distribuídos reside na forma como o hardware se conecta ao software. Nos sistemas de laboratório, geralmente são utilizadas interfaces de computador, como placas PCI ou comunicação serial. Já os sistemas de DAQ distribuídos utilizam comunicação em rede. Por fim, os sistemas de DAQ portáteis são unidades pequenas e leves, facilmente transportáveis manualmente.

^y Autor correspondente.

Endereço de e-mail: pasquati@ifsp.edu.br (RP Pontarolli).

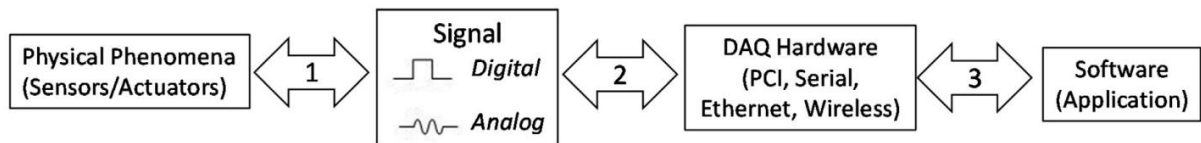


Figura 1. Diagrama de blocos do sistema de aquisição de dados.

Os equipamentos de aquisição de dados (DAQ) são produtos comerciais tradicionais com um custo considerável. E, por vezes, é necessário adquiri-los. sinais específicos que não são suportados pelo hardware DAQ padrão. Consequentemente, é comum encontrar na literatura artigos que desenvolvem hardware DAQ de baixo custo e de código aberto para diversas aplicações diferentes [5,6].

Analisando o cenário de sistemas de aquisição de dados (DAQ) para a indústria, na maioria dos casos são utilizados sinais analógicos e digitais padronizados. como sinais analógicos de 4–20 mA e 0–10 VCC, e sinais digitais de 24 VCC. Um elemento importante em sistemas industriais para a aquisição desses sinais é a E/S remota (entrada/saída), responsável pela aquisição de dados das entradas/sensores e saídas/atuadores (sinais analógicos e digitais) de um processo industrial. Na automação hierárquica tradicional ISA-95, arquitetura [7], essas E/S remotas são assim denominadas precisamente porque estão fisicamente em um local remoto a uma certa distância do controlador. E ambos estão conectados e se comunicam por meio de um meio físico, como pode ser visto na Fig. 2 a), em um cenário com apenas um controlador [8]. A E/S distribuída é uma evolução da E/S remota, Fig. 2 b) e também está em um certo distância do controlador, mas possui alguma capacidade de processamento e compartilha uma rede, geralmente um protocolo industrial, com Outros controladores e módulos de E/S distribuídos.

Os processos industriais, assim como muitas outras infraestruturas industriais, dependem do Sistema de Supervisão, Controle e Aquisição de Dados (SCADA). Sistemas de Controle de Análise e Adaptação de Sistemas (SCADA) e Sistemas de Controle Distribuído (DCS) para executar suas funcionalidades complexas. A Internet Industrial das Coisas A IIoT é complementar a esses sistemas, utilizando informações de SCADA e DCS como fontes de dados. O SCADA concentra-se no monitoramento e controle, enquanto a IIoT busca conectividade para coletar e analisar dados para melhorar a tomada de decisões e aumentar a produtividade [9].

As novas aplicações relacionadas à IIoT e à Indústria 4.0 (I4.0) devem evoluir em termos de soluções de automação para superar os desafios. Os desafios da descentralização, modularidade e independência dos sistemas. Isso significa, por exemplo, que novas arquiteturas e tecnologias de comunicação são necessárias para facilitar a associação de soluções de automação, como E/S distribuídas, controladores programáveis, SCADA e sistemas de gerenciamento. Conforme apresentado em [10], a inovação técnica da Indústria 4.0 Baseia-se na integração vertical e horizontal de sistemas industriais, com engenharia digital contínua em todo o produto. ciclo de vida e descentralização de recursos computacionais. Para que isso ocorra, é necessário um modelo comum que agrupe essas condições. dentro de uma arquitetura de referência foi definida para a Indústria 4.0 [11], chamada RAMI 4.0 (Modelo Arquitetônico de Referência para a Indústria 4.0). O RAMI 4.0 é baseado em uma Arquitetura Orientada a Serviços (SOA).

A Arquitetura Orientada a Serviços (SOA) é um paradigma recente para soluções de automação com potencial comprovado para fornecer integração vertical e interoperabilidade para sistemas em rede por meio do uso e compartilhamento de serviços. A tendência mais recente em SOA para aplicações industriais são os microsserviços. Uma variante da SOA, os microsserviços são um estilo arquitetural no qual as aplicações são decompostas em componentes. pequenos serviços, oferecendo modularidade, tornando os aplicativos mais fáceis de desenvolver, testar, implantar e, mais importante, modificar. manter [12]. A referência [13] define microsserviços como pequenos serviços independentes que trabalham em conjunto. Esses serviços executam Em seus próprios processos, eles se comunicam por meio de mecanismos síncronos (solicitação/resposta) e assíncronos (solicitação/retorno de chamada), geralmente com integração web via REST.

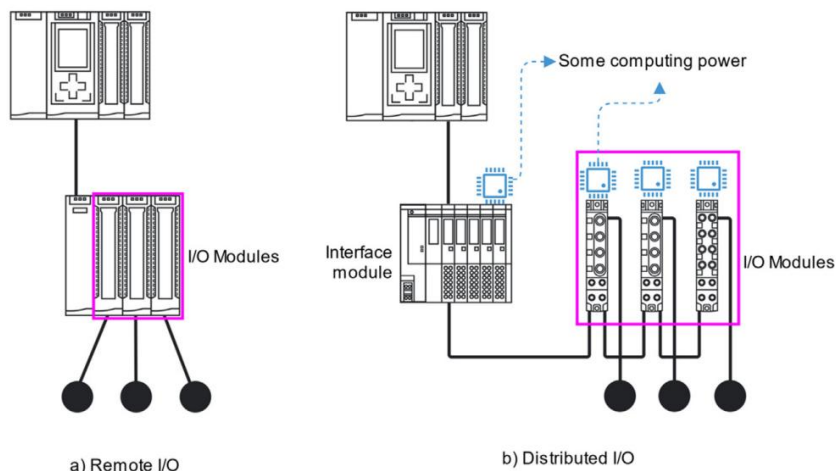


Figura 2. Representação de dispositivos de E/S em aplicações de automação industrial (Adaptado de [8]).

Os microsserviços são implantados e escalados de forma independente, possuem limites bem definidos e podem ser escritos de forma personalizada. em diferentes idiomas e utilizam diferentes recursos para armazenamento de dados. Uma comparação entre arquiteturas monolíticas e A arquitetura de microsserviços é apresentada na [Figura 3 \[14\]](#). Como pode ser observado, o maior ganho na adoção de microsserviços é a escalabilidade. Observa-se, no lado esquerdo, que um ambiente construído com base em uma abordagem monolítica dificulta a escalabilidade de serviços específicos, enquanto, no lado direito, os serviços podem ser clonados para facilitar o aumento do acesso a eles, diminuindo a carga. solicitações no mesmo servidor, além da capacidade de lidar com falhas. Os recursos podem ser melhor aproveitados, evitando sobrecarga de recursos. desperdício com alocação de serviços não utilizada ou subutilizada.

O projeto Architecture for Service-Oriented Process (IMC-AESOP) [\[15\]](#) desenvolveu inicialmente uma SOA em nuvem para aplicações de automação industrial, com foco principal em aplicações de controle de processos, sistemas SCADA e DCS. O grande avanço da SOA A arquitetura era a disponibilidade de serviços criados em uma infraestrutura de computação em nuvem, o que aumentou a arquitetura. disponibilidade e permitiu que se adaptasse às variações nas aplicações devido à replicação ou criação de novos serviços em nuvem. Vários serviços foram desenvolvidos para suportar todas as funcionalidades propostas pela arquitetura, como monitoramento, controle, serviços de alarme, simulação e supervisão [\[16\]](#).

Neste projeto, foram utilizadas E/S distribuídas tradicionais dentro de um serviço mediador criado para obter os dados da E/S distribuída legada e replicá-los para os serviços SOA. A referência [\[17\]](#) apresenta essa estrutura de DAQ, como pode ser visto na Figura 4 .

Neste caso, o Gateway controla um conjunto de dispositivos de camada inferior que não possuem suporte a serviços, cada um dos quais é exposto através do Gateway como um dispositivo habilitado a serviços. Isso permite que eles substituam gradualmente os dispositivos legados por dispositivos habilitados a serviços web nativamente. sem afetar os aplicativos que utilizam esses dispositivos.

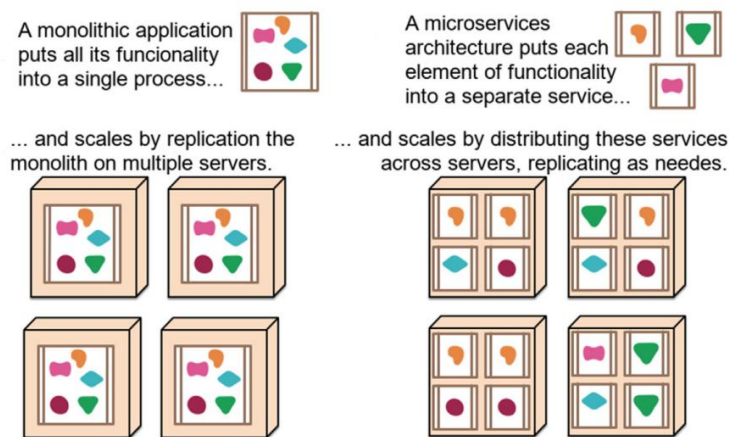


Figura 3. Comparação de estruturas monolíticas e microsserviços [\[14\]](#).

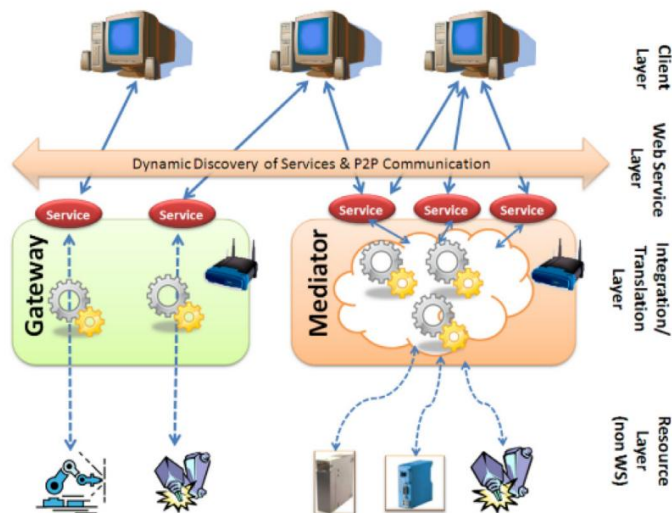


Figura 4. Conceitos de Gateway vs Mediator de Serviço [\[17\]](#).

Ainda no mesmo projeto, um mediador pode conter múltiplas fontes de dados, que podem ou não ser processadas antes de serem disponibilizadas como um serviço. A referência [17] utilizou o mediador para agregar diversos dispositivos não habilitados para serviço, onde Aplicações nas camadas superiores podem se comunicar com o mediador em vez de usar interfaces proprietárias.

Uma arquitetura orientada a microsserviços (MOA) para automação e controle em cenários da Indústria 4.0 foi desenvolvida em [18,19] pelos autores, que se concentraram no desenvolvimento de microsserviços para controle de processos. Seguindo essa tendência de uso Este artigo descreve o desenvolvimento de um sistema de E/S distribuído de código aberto como um microsserviço, considerando a automação industrial e as aplicações da Indústria 4.0. Ao contrário do desenvolvimento de E/S remota tradicional [20,21] (Fig. 2) ou de um serviço mediador, este sistema propõe uma abordagem diferente. [17] (Fig. 4) para DAQ industrial, nossa proposta implementa uma E/S distribuída diretamente conectada à rede orientada a serviços. como mostrado na Figura 5. Consequentemente, todas as informações adquiridas da E/S distribuída ficam automaticamente disponíveis para uso por qualquer sistema. outro serviço ou aplicação de forma padronizada e interoperável.

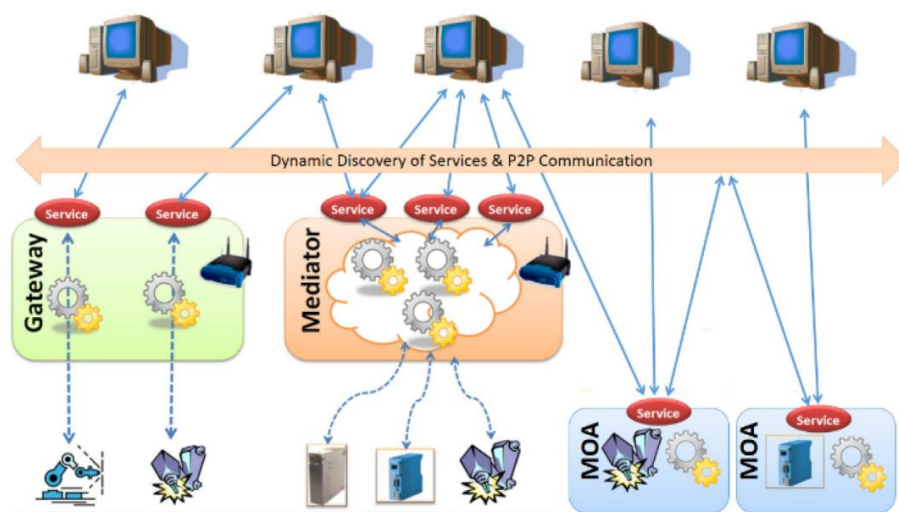


Figura 5. Comparação de conceitos de DAQ para SOA industrial: Gateway vs Serviço Mediador vs Serviço de E/S Distribuído (Adaptado de [15]).

Estrutura molecular

Para a criação do microsserviço de E/S distribuída, utilizamos o framework Molecular. Um framework representa uma base Uma estrutura ou plataforma de desenvolvimento, contendo ferramentas, bibliotecas, sistemas e componentes que suportam o processo de desenvolvimento de uma solução específica. O framework Molecular é um framework de desenvolvimento que utiliza a linguagem JavaScript para desenvolver soluções aplicações orientadas a microsserviços [22]. O framework é de código aberto e roda na plataforma Node.js, na qual ser-

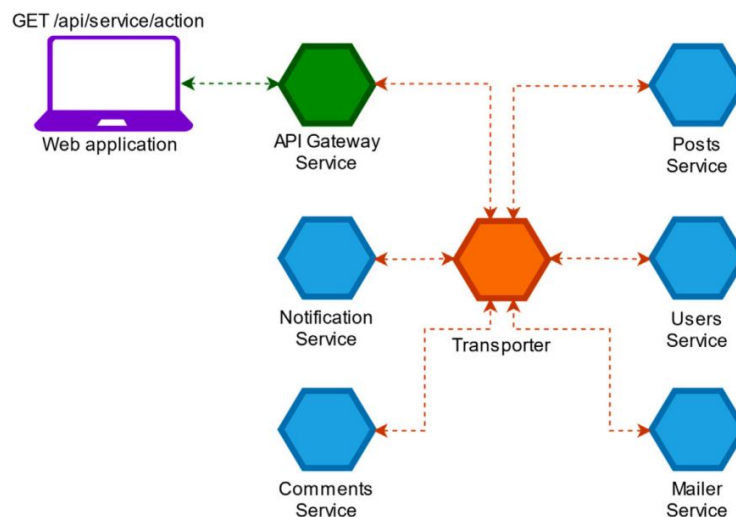


Figura 6. Arquitetura de microsserviços do Molecular Framework.

As tarefas são executadas em nós individuais que se comunicam automaticamente por meio do serviço de comunicação (Transporter). Neste caso, Em uma arquitetura distribuída, os serviços podem ser facilmente replicados para proporcionar resiliência e tolerância a falhas.

Na arquitetura Molecular, Fig. 6, todos os microsserviços são iguais, sem hierarquia ou prioridade. Uma grande vantagem de Essa estrutura prevê que todos os microsserviços desenvolvidos tenham um recurso de registro e descoberta automático disponível. Dessa forma, todos os serviços existentes são informados após a criação de um novo serviço ou a disponibilidade de novas funcionalidades em um serviço. Outra característica importante é o balanceamento de carga automático, que tem a função de distribuir dinamicamente a carga de comunicação entre os microsserviços replicados. Cada nó pode conter um ou mais microsserviços e é responsável por gerenciar seus próprios dados e seu próprio banco de dados. Os microsserviços podem oferecer e executar diferentes tarefas, chamadas de ações.

Na Molecular, o serviço Transporter gerencia automaticamente a comunicação entre microsserviços. A grande vantagem é que ele gerencia a comunicação entre os microsserviços. É a separação entre produtores e consumidores de eventos e dados. A separação simplifica o desenvolvimento e aumenta a produtividade. disponibilidade em comparação com o uso de transações distribuídas. Se nenhum consumidor estiver disponível para processar um evento, o Transporter irá Coloque o evento em fila até que a mensagem possa ser processada.

O serviço Transporter resolve a comunicação entre sistemas completamente diferentes de forma confiável. O broker de mensagens possibilita a integração desses sistemas para que possam trocar dados de maneira independente. A troca de A comunicação entre microsserviços que utilizam o Transporter é realizada pelo módulo Serializers (por exemplo, JSON). Responsável pela serialização dos dados da mensagem.

Descrição do hardware

Considerando a atual migração da automação industrial para arquiteturas distribuídas como SOA e MOA, a contribuição deste artigo é detalhar o desenvolvimento de E/ S distribuída como um microsserviço para aquisição de dados industrial. A solução é composto por um computador de placa única (Raspberry Pi) com um módulo de expansão de E/S industrial (placa industrial MegaIO), um framework de microsserviços (Molecular) e um software personalizado incorporado no hardware (ver Tabela 1). O distribuído A interface de E/S é de código aberto, de baixo custo e simplifica a aquisição de dados de E/S para aplicações industriais. A comunicação em rede com a E/S distribuída é transparente e automática (gerenciada pelo Transporter), o que significa que não é necessário nenhum tipo de codificação de rede. Essa comunicação padronizada proporciona interoperabilidade e escalabilidade para qualquer sistema de automação industrial. aplicações que precisam usar dados adquiridos pela E/S distribuída. Além disso, a E/S distribuída também possibilita aplicações IIoT, pois fornece automaticamente acesso à web por meio de uma API HTTP(S) para todas as E/S disponíveis. Alguns dos usos possíveis são:

Os itens fornecidos para este equipamento são:

- a) Coletar sinais de entrada analógicos e digitais de sensores;
- b) Atualizar os sinais de saída analógicos e digitais para os atuadores;
- c) Habilitar redundância automática na aquisição de dados de E/S;
- d) Padronizar a comunicação remota em rede com dispositivos conectados às E/S;
- e) Habilitar o acesso web à E/S através do API Gateway;
- f) Fornecer dados de E/S industriais para aplicações de automação (por exemplo, registro de dados de E/S em um histórico de processos, um processo) circuito de controle, aquisição de dados para um sistema SCADA).

Tabela 1
Especificações de E/S distribuída.

Tabela de especificações de E/S distribuída	
Nome do hardware	E/S distribuída
Área temática	Engenharia e automação Medições e aquisição de dados do setor Ferramentas educacionais e alternativas de código aberto para a infraestrutura existente. Instrumentação
Tipo de hardware	Medições e comunicação em campo
Análogo comercial mais próximo	UniPi [1] Revolução Pi [2] Iono Pi [3] Chapéu Monarco [4]
Licença de código aberto	Licença Pública Geral GNU (GPL) 3.0.
Custo do hardware	\$ 152,47
Repositório de arquivos de origem	https://doi.org/10.17605/OSF.IO/EG9FU

Resumo dos arquivos de projeto

- 1. Esquema de hardware: HAT empilhável de 8 camadas para Raspberry Pi (ver Tabela 2).
- 2. Microsserviços em linguagem JavaScript criados utilizando o framework Molecular para acessar as funcionalidades de HAT empilhável de 8 camadas para automação industrial, compatível com hardware Raspberry Pi, de forma distribuída.

Tabela 2

Resumo dos arquivos de projeto.

EU IA	Nome do arquivo de projeto	Tipo de arquivo	Licença de código aberto	Localização do arquivo
1	megaind-rpi.pdf	Esquemas	GPL 3.0	https://osf.io/v5tz3/
2	megaind-rpi.service.js	Software	GPL 3.0	https://osf.io/gd2bf/
3	api.service.js	Software	MIT	https://osf.io/zg86k/
4	nats 2.1.7	Software	Apache 2.0	https://hub.docker.com/_/nats
5 6	Raspberry Pi OS	Software	GPL	https://www.raspberrypi.org/software/
	Node.js	Software	MIT	https://nodejs.org/en/download/

- 3. Microserviços em linguagem JavaScript criados utilizando o framework Moleculer com a função de publicar os microserviços, que podem ser acessados através de requisições HTTP pelo navegador, por exemplo.
- 4. Servidor de mensagens, transportador, responsável pela troca de mensagens entre microserviços.
- 5. Sistema operacional para o computador de placa única Raspberry Pi;
- 6. Software de código aberto para executar aplicações JavaScript.

Lista de materiais

Instruções de compilação para E/S distribuída

A parte de hardware da E/S distribuída está disponível comercialmente no mercado (ver [Tabela 3](#)). Ela é composta por um único computador de placa Raspberry Pi e uma placa de E/S de automação industrial para Raspberry Pi [\[23\]](#). A parte mecânica da construção é muito simples, na qual o kit de placa de E/S de automação industrial (megaind) (Seção "E/S de automação industrial (megaind)" O kit de cartão" é montado em cima do Raspberry Pi, conforme mostrado na [Fig. 7](#).

Tabela 3

Lista de materiais.

Componente	N. Unidade	Fonte dos materiais	Material tipo
Raspberry Pi 3	1 1	US\$ 35,00 https://www.raspberrypi.com/products	Eletrônico
SanDisk 16 GB		US\$ 6,59 https://www.amazon.com/SanDisk-Ultra-SDSQUINS-016G-GN3MN-UHS-I-microSDHC/dp/B074B4P7KD	Eletrônico
Fonte de energia 24 V	1	US\$ 18,98 https://www.amazon.com/AVAWO-Switching-Transformer-Regulated-Computer/dp/B0146IAXYO	Eletrônico
Cartão Megaind	1	US\$ 90,00 https://sequentmicrosystems.com/collections/industrial-automation/products/industrial-framboesa-pi	Eletrônico
CR2450	1 unidade por	US\$ 1,90 https://www.amazon.com/GloFX-CR2450-Lithium-Battery-Pack/dp/B07ZQNHPZC	Eletrônico
Custo total		\$ 152,47	



Figura 7. Hardware da E/S distribuída [\[23\]](#).

Kit de placas de E/S para automação industrial (megaind)

A placa de E/S de automação industrial (megaind), Fig. 8, é composta pelos seguintes itens nas Fig. 9 e Fig. 10.

Quatro espaçadores macho-fêmea de latão M2,518 mm; quatro parafusos de latão M2,55 mm; quatro porcas de latão M2,5; ferragens de montagem; dois jumpers de nível de pilha; quatro conectores fêmea de 8 pinos para E/S.



Fig. 8. Placa de automação industrial para Raspberry Pi [23].



Fig. 9. Hardware de montagem e jumpers [23].



Fig. 10. Conectores para todas as entradas e saídas [23].

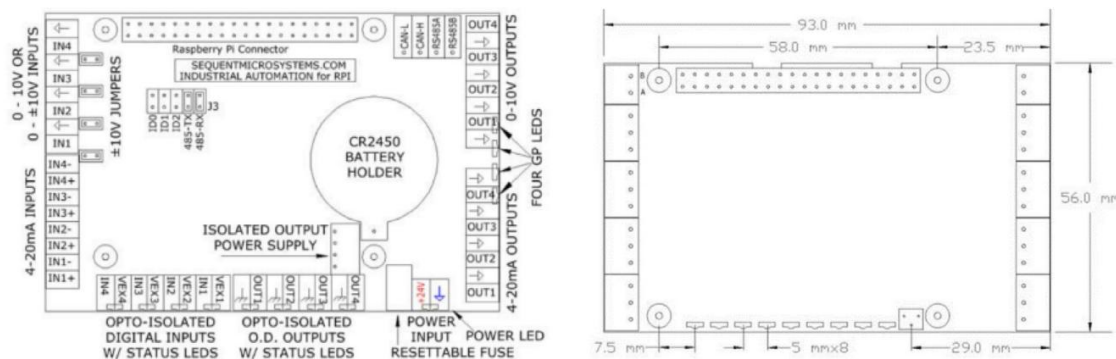


Figura 11. Placa de automação industrial para layout da placa Raspberry Pi e especificações mecânicas [23].

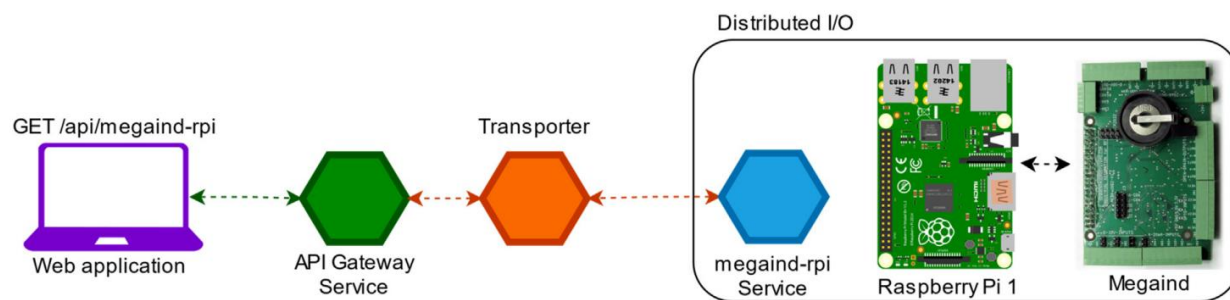


Figura 12. E/S distribuída com acesso externo.

Um conector fêmea de 4 pinos para portas RS465.

Um conector fêmea de 2 pinos para alimentação.

Características da placa de E/S para automação industrial (megaind)

O layout e as especificações mecânicas da placa de E/S de automação industrial (megaind) são mostrados na [Fig. 11](#).

As principais características da placa de E/S para automação industrial (megaind) são:

Quatro entradas digitais opticamente isoladas com LEDs indicadores de status; quatro entradas analógicas de 12 bits, 0–10 V ou ± 10 V; quatro entradas de 12 bits, 4–20 mA, opticamente isoladas; quatro saídas de dreno aberto, 24 V/4 A, opticamente isoladas; quatro saídas analógicas de 12 bits, 0–10 V; quatro saídas analógicas de 12 bits, 4–20 mA; quatro LEDs de uso geral; proteção TVS em todas as entradas; watchdog de hardware integrado; fusível rearmável integrado; relógio de tempo real com bateria de reserva.

Informações adicionais:

Comunicação: conecte a placa a outros sistemas de Automação Industrial usando as portas de comunicação RS485/Modbus e CAN; Compatibilidade: a placa de E/S para automação industrial compartilha o barramento I2C usando apenas dois dos pinos GPIO do Raspberry Pi para gerenciar todas as oito placas. Esse recurso deixa os 24 GPIOs restantes disponíveis para o usuário. É compatível com todas as versões do Raspberry Pi, do Zero ao 4, e possui todas as E/S necessárias para seus projetos de Automação Industrial.

Requisitos de alimentação: A placa de Automação Industrial precisa de 24 VCC para funcionar e deve ser alimentada por meio de seu próprio conector. A placa fornece 5 V e até 3 A para o Raspberry Pi no barramento GPIO.

Conectores plugáveis: Todas as E/S são conectadas a conectores plugáveis de alta resistência (8A) com espaçamento de 3,5 mm, o que torna a fiação em campo muito conveniente para instalação e depuração.

Empilhamento de várias placas: É possível empilhar até oito placas de Automação Industrial em um Raspberry Pi. As placas podem ser instaladas em qualquer ordem. Cada placa é identificada por jumpers que indicam o nível na pilha.

Instruções operacionais e testes

Para explicar as instruções de operação e os testes iniciais, bem como o aprendizado de uso do microserviço de E/S distribuída, um cenário simples de aplicação web é apresentado na [Figura 12](#). Nesse cenário, os dados de E/S da E/S distribuída são acessados remotamente pela web (navegador de internet) usando requisições de API (método HTTP GET). Todos os microserviços (API Gateway, Transporter e E/S Distribuída) foram implantados em um único hardware (Raspberry Pi) para simplificar as instruções de operação e os recursos necessários. No entanto, seria possível distribuir esses microserviços em três hardwares diferentes (por exemplo, com cada serviço em um Raspberry Pi distinto).

Nesse cenário, o microserviço de E/S distribuída fornece acesso às E/S (sensores/atuidores) conectadas à placa de automação. O microserviço Transporter fornece comunicação de rede com outros microserviços (por exemplo, E/S distribuída). O microserviço de gateway de API fornece acesso web HTTP(s) aos dados de E/S da E/S distribuída. Uma lista resumida contendo todas as ações disponíveis do serviço megaind-rpi para aquisição de dados de E/S da placa megaind é apresentada na [Tabela 4](#).

Tabela 4
Resumo das ações do microserviço de E/S distribuída para aquisição de dados de E/S.

N. Ação	Descrição	parâmetros	Retornar
1 conjunto de tensão de saída	Escreva o valor da tensão de saída de 0 a 10 V (V).	Pilha, 3 canais, 4 valor5	0 (Sucesso) ou 1 (Erro)
2 obterTensãoDeSaída	Leia o valor da tensão de saída de 0 a 10 V (V)	pilha, pilha de	Porcentagem de 0 a 10 V [0..100]
obterTensãoDeEntrada	Leia o valor da tensão de entrada de 0 a 10 V (V)	canal, pilha de	Porcentagem de 0 a 10 V [0..100]
3 4 definirCorrenteDeSaída	Escreva o valor de saída de 4–20 mA (mA)	canal, canal, valor6	0 (Sucesso) ou 1 (Erro)
5 obterAtualDeSaída	Leitura de 4–20 mA Valor da corrente de saída (mA)	pilha, canal	Porcentagem de 4 a 20 mA [0..100]
6 obterEntradaAtual	Leia o valor de entrada de 4–20 mA (mA)	pilha, canal	Porcentagem de 4 a 20 mA [0..100]
7 setOutputOpenDrain	Escrever valor PWM de saída de dreno aberto (0..100%)	pilha, canal, valor7	0 (Sucesso) ou 1 (Erro)
8 getOutputOpenDrain	Lê o valor PWM de saída de dreno aberto (0..100%)	pilha, canal	Porcentagem de PWM de 0 a 24 V [0..100]
9 obterOpto de Entrada	Leia o status do optometrista	pilha, canal	0 (DESLIGADO) ou 1 (LIGADO)
10 obterDiagnóstico da Placa	Exibir informações sobre a placa, versão do firmware, fonte de alimentação e temperatura	Pilha	firmware: versão, potência: tensão (V) ,temperatura: CPU (C)

³ Nível de empilhamento do cartão megaind (selecionável por jumpers de endereço [0..7]). O cartão megaind pode ser empilhado até 8 cartões no mesmo Raspberry Pi.
⁴ Número do canal selecionado [1.0.4] da E/S da placa megaind.
⁵ Porcentagem de 0 a 10 V [0..100].
⁶ Porcentagem de 4 a 20 mA [0..100].
⁷ Porcentagem de 0 a 24 V PWM [0..100].

Para cada uma dessas ações, existe uma requisição HTTP(s) padronizada. Por exemplo, a ação setOutputVoltage é responsável por escrever (atualizar) um valor analógico de 0 a 10 V em um canal selecionado da placa megaind. Uma requisição genérica para O acesso a essas ações é padronizado no seguinte formato (endereço-ip1 :porta2).

```
https://ip-address:port/megaind-rpi.action?  
Cabeçalho {"Content-Type": "application/json".  
Corpo {"param-100: valor, "param-200: valor}.
```

Hardware

Conecte o cartão em cima do seu Raspberry Pi e ligue o sistema, Fig. 7.
Habilite a comunicação I2C no Raspberry Pi usando o raspi-config2 .

Software

Prepare o Raspberry Pi com o software abaixo. Para obter informações detalhadas, visite o link disponível na Seção-
ção 3 Resumo dos arquivos de projeto.

```
Sistema Operacional Raspberry Pi  
Node.js  
transportador NATS  
api.service.js  
megaind-rpi.service.js
```

Uso

O procedimento de utilização (passo a passo) está disponível em <https://github.com/pontarolli/megaind-rpi>. Aqui é mostrado um exemplo simples. Exemplo de escrita de um novo valor (atualização) em um canal de saída analógica da E/S distribuída. Por exemplo, a saída de um A saída de tensão analógica da E/S distribuída é definida para 2,5 V (25% da escala) usando a solicitação setOutputVoltage na Fig. 13. Depois disso, é possível confirmar o valor de saída com um multímetro, como mostrado na Fig. 14.

¹ Hardware (Raspberry Pi) no qual o serviço de gateway de API está em execução.
² Porta TCP na qual o serviço API Gateway está escutando.

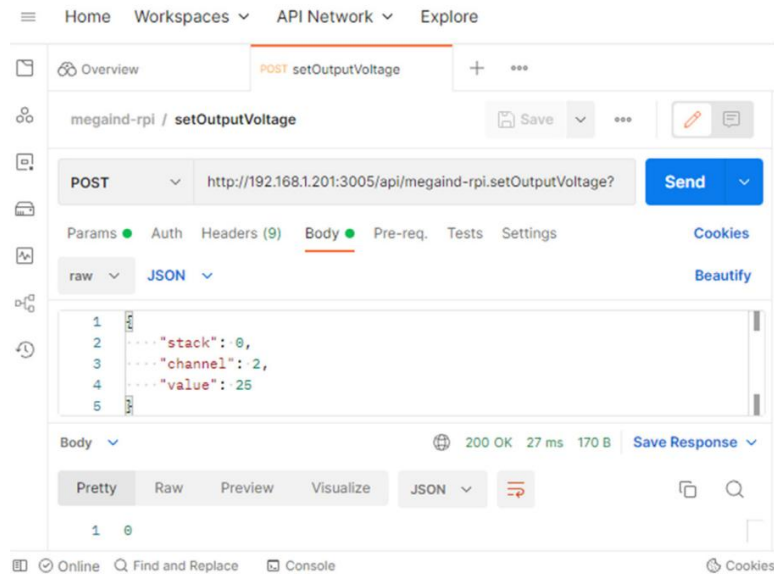


Figura 13. Exemplo de uma requisição de ação setOutputVoltage usando o software Postman.

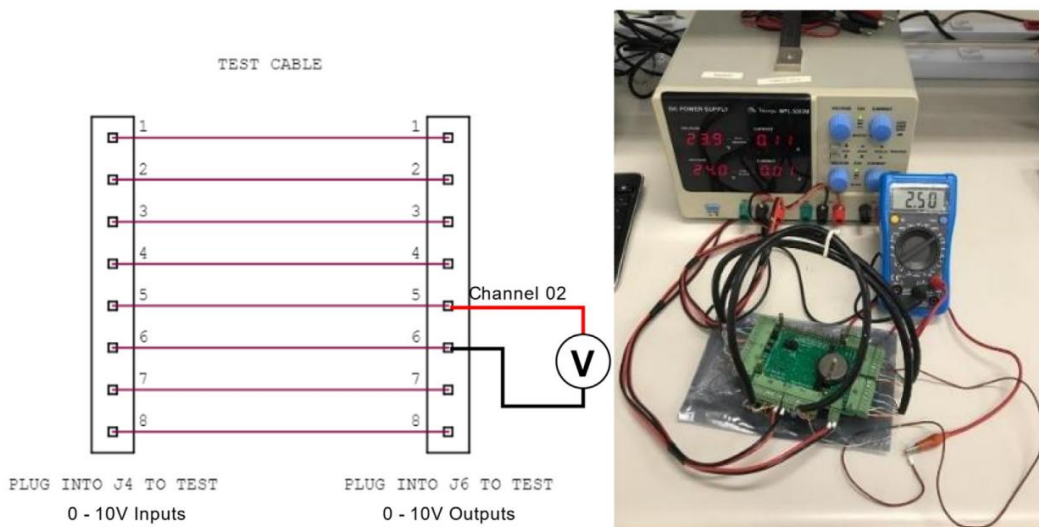


Figura 14. Teste de operação da E/S distribuída: atualização da saída de tensão analógica.

<https://localhost:3000/megaind-rpi.setOutputVoltage?>
 Cabeçalho {"Content-Type": "application/json"}.
 Corpo {"pilha": 0, "canal": 2, "valor": 25}.
 Retorno: 0 (Sucesso) ou 1 (Erro).

Também é possível realizar um autoteste conectando as saídas de tensão das E/S distribuídas às suas respectivas entradas de tensão, conforme mostrado na Figura 14 (esquemas disponíveis em <https://github.com/pontarolli/megaind-rpi/tree/master/schematics>). página 4/16). Após configurar as tensões de saída, seria possível obter o valor nas entradas usando a ação 'getInputVoltage'. O autoteste facilita os testes, pois não requer hardware adicional (sensores/atuadores).

Caso seja necessário acessar remotamente as entradas e saídas do sistema de E/S distribuído, por exemplo, usando um aplicativo Web ou pela Internet, utiliza-se o serviço API Gateway. Conforme explicado, o serviço API Gateway cria automaticamente uma rota HTTP(s) para solicitar qualquer um dos recursos de E/S disponíveis no sistema de E/S distribuído. É possível enviar solicitações HTTP.

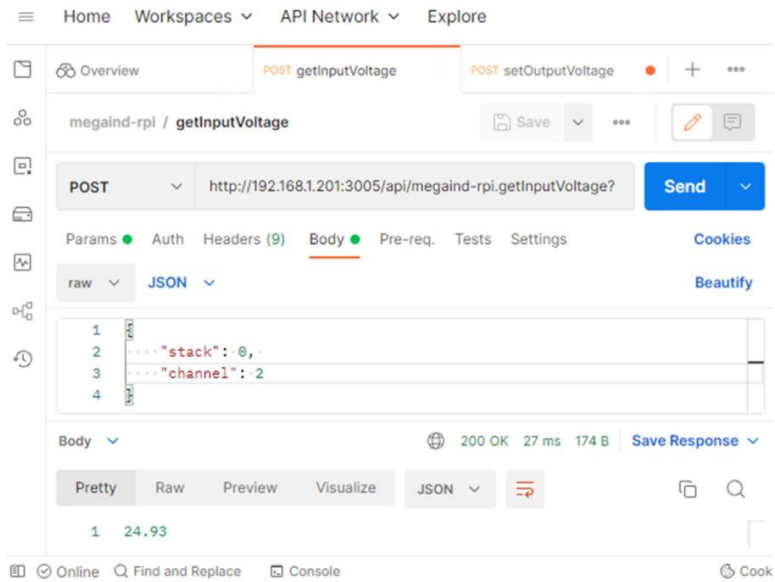


Figura 15. Exemplo de uma requisição de ação getInputVoltage usando o software Postman.

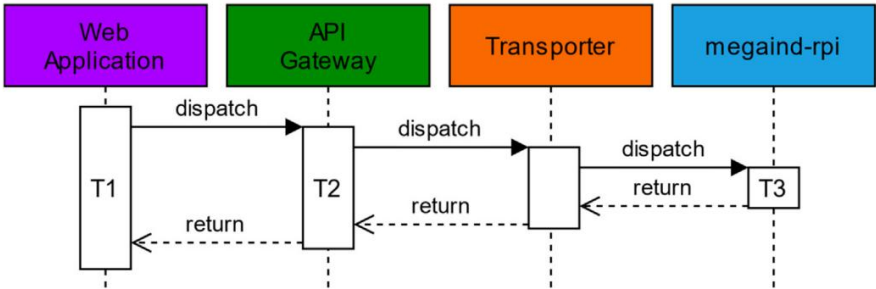


Figura 16. Diagrama de sequência para aplicação externa.

com qualquer aplicação, como o Postman mostrado na Fig. 15, que exibe uma solicitação de leitura de entradas de tensão analógica do E/S distribuída.

<https://localhost:3000/megaind-rpi.getInputVoltage?>
Cabeçalho {"Content-Type": "application/json"}.
Corpo {"pilha": 0, "canal": 2}.
Retorno: Percentagem de 0 a 10 V [0..100].

A sequência de execução desta solicitação pela aplicação web pode ser vista no diagrama de sequência abaixo, na Figura 16. A solicitação GET api/megaind-rpi passa primeiro pelo serviço API Gateway e, em seguida, pelo serviço Transporter. até chegar ao serviço de E/S distribuído e então retornar.

O tempo médio total de requisição foi de 22,9 ms, assim como o tempo em cada parte da rota de comunicação, que são mostrados a seguir. na tabela da Tabela 5.

Tabela 5
Tempos de comunicação (T na Fig. 16) para uma solicitação de API da E/S distribuída.

T	Ação	Média (ms)	Mínimo (ms)	Máximo (ms)	Desvio padrão (ms)	Taxa de sucesso (%)
1	Aplicativo web	22,9	16,3	111,5	22,3	99,2
2 3	Gateway de API megaind-rpi	19,0	13,5	346,6	41,7	
		4,8	2,7	22,7	1,2	

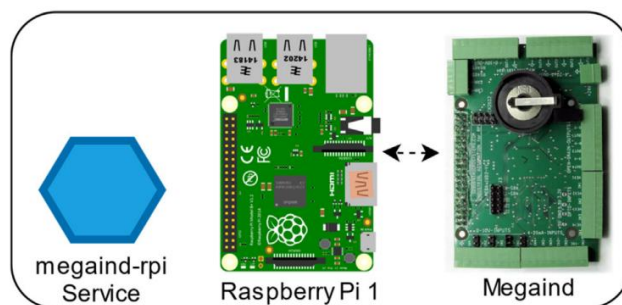


Figura 17. E/S distribuída.

Exemplos de aplicação

Na E/S distribuída da [Figura 17](#), o serviço `megaind-rpi` é representado por um hexágono azul. Este serviço implementa a codificação para aquisição de dados (DAQ) de E/S industrial usando a placa de automação Megaind. A interface entre o Raspberry Pi que executa o serviço e a placa Megaind é feita via protocolo I2C (Inter Integrated Circuit).

Os microsserviços são como peças de LEGO, e criar uma aplicação é como compor microsserviços ou usar diferentes microsserviços de acordo com a sequência necessária. Consequentemente, a E/S distribuída como microsserviço desenvolvida neste artigo pode ser usada como base para qualquer outra aplicação científica ou de pesquisa que precise adquirir dados. No cenário industrial abordado neste artigo, apresentaremos exemplos como um sistema de histórico de processos, um controlador de processos em malha fechada ou mesmo uma aplicação de IIoT (Internet Industrial das Coisas) que acessa dados da E/S distribuída.

Exemplo 1 – Aplicação interna

Existem dois tipos de aplicações que podem ser criadas usando a arquitetura Molecular: aplicações internas e aplicações externas. Uma aplicação é uma funcionalidade de alto nível que utiliza microsserviços. As aplicações internas são todas programadas em JavaScript usando Node.js, e os serviços se comunicam entre si por meio do Transporter, representado pelo hexágono laranja na [Figura 18](#). Nesse caso, um serviço de aplicação, representado por um hexágono roxo, pode solicitar os dados de uma entrada/saída (I/O) da E/S distribuída usando o Transporter. O Transporter localiza a E/S distribuída (serviço `megaind-rpi`) e retorna o valor solicitado (dados de um sensor, por exemplo).

Exemplo 2 – Aplicação externa e réplica

As aplicações externas podem ser implementadas em qualquer tipo de plataforma, e a comunicação com os serviços internos utiliza a comunicação HTTP (REST) em conjunto com o serviço API Gateway representado por um hexágono verde na [Figura 19](#). O gateway de API faz a ponte entre os dados externos e internos dos microsserviços do Molecular.

Neste caso, os dados de uma entrada/saída (I/O) da E/S distribuída são solicitados por meio de um método HTTP GET a partir de uma aplicação web. A requisição do API Gateway é enviada ao Transporter, que localiza a E/S distribuída (serviço `megaind-rpi`) e retorna o valor solicitado (dados de um sensor, por exemplo).

O exemplo na [Figura 19](#) também apresenta um cenário em que a redundância de aquisição de dados de E/S industrial é necessária. A E/S distribuída como um microsserviço permite a redundância na aquisição de dados de E/S por meio da replicação do hardware e do microsserviço. Neste caso,

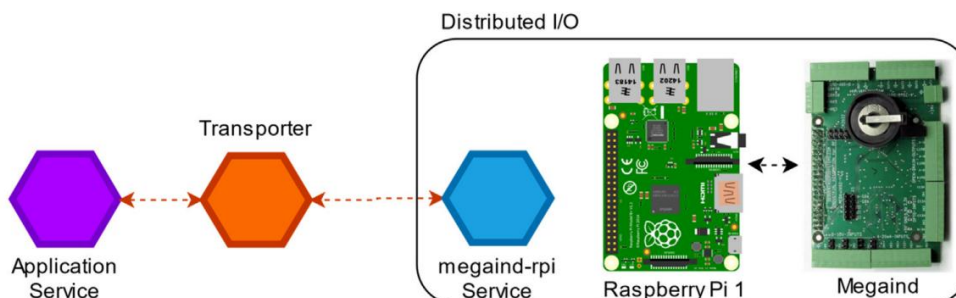


Figura 18. Exemplo de aplicação interna usando E/S distribuída.

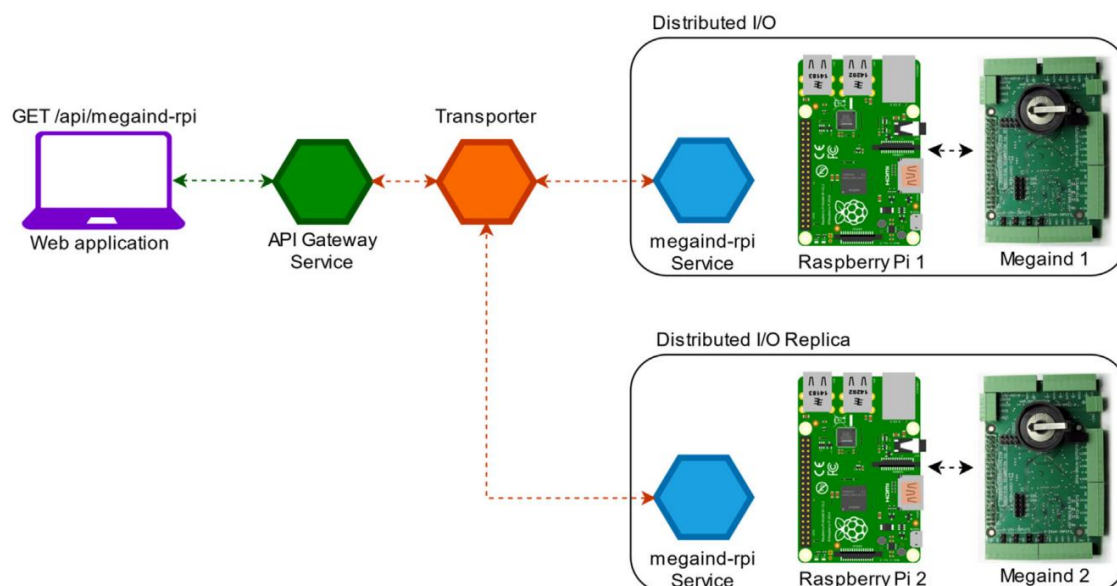


Figura 19. Exemplos de aplicação externa e redundância usando E/S distribuída.

Em um cenário de redundância, há mais de uma E/S distribuída conectada à mesma E/S. Os microsserviços replicados (megaind-rpi) habilita automaticamente a redundância, de forma que, se uma das E/S distribuídas falhar, as outras estarão disponíveis para responder.

Exemplos de redundância de E/S de aquisição de dados industriais podem ser encontrados em grandes fabricantes de automação, como mostrado em [24], Fig. 20, onde a E/S distribuída comercial (modelo ET200M) é instanciada duas vezes para o mesmo controlador (S7-400H). Nessa configuração, se uma das E/S distribuídas falhar, a outra estará disponível para adquirir o valor do sensor. No entanto, é obrigatório

A codificação de rede é necessária para implementar a redundância. Além disso, essas soluções comerciais para redundância em aquisição de dados têm um custo elevado e não são de código aberto. No entanto, para projetos de pesquisa e desenvolvimento, a solução

A solução proposta neste artigo seria mais viável em termos de custos, oferecendo a mesma funcionalidade de redundância com uma implementação simplificada.

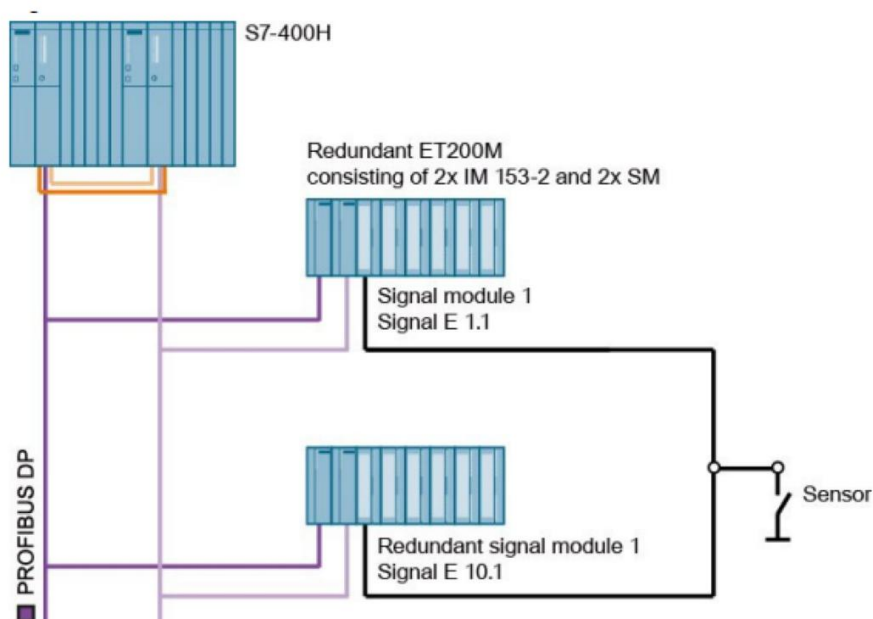


Figura 20. E/S Redundante Distribuída Comercial [24].

Exemplo 3 – Controlador de processo em circuito fechado

Para demonstrar um exemplo de controlador de processo em malha fechada, foi criado um serviço de composição, representado por um hexágono roxo na [Figura 21](#). Essa composição solicita, de forma cíclica e sequencial, os seguintes microsserviços: E/S distribuída (serviço megaind-rpi: entrada do sensor), serviço de controle (algoritmo de controle) e E/S distribuída (serviço megaind-rpi: saída do atuador). O serviço de composição define o valor ou perfil de referência, bem como a taxa cíclica. Este exemplo de controle em malha fechada foi apresentado pelos autores em [\[19\]](#), onde é investigada uma aplicação real utilizando a composição de microsserviços para controle de processos em uma planta piloto de processo industrial.

Este exemplo apresenta uma aplicação de pesquisa do serviço de E/S distribuído desenvolvido neste artigo. A planta piloto industrial mostrada na [Figura 22](#) foi desenvolvida pelos autores em conjunto com a empresa Emerson Automation Solutions para viabilizar a pesquisa em aplicações de microsserviços para automação e controle. A planta piloto possui um tanque principal de aço inoxidável na parte inferior para armazenamento de água, e o líquido pode ser bombeado para o topo da planta por duas rotas diferentes: para o acrílico

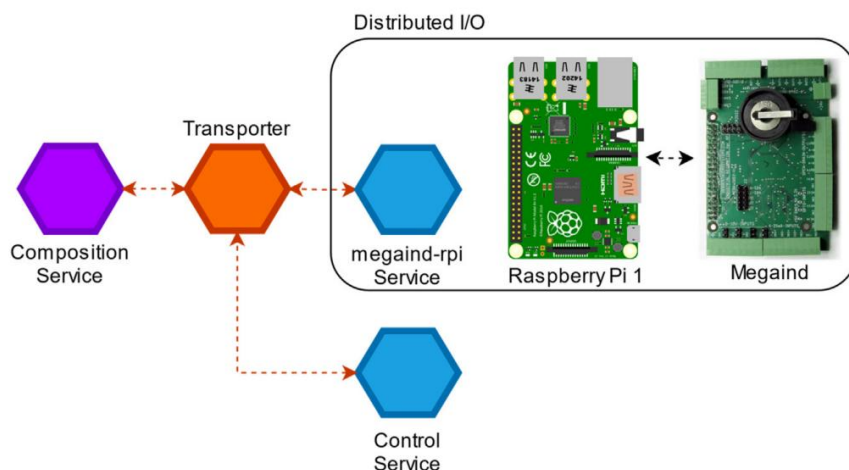


Figura 21. Exemplo de composição de aplicação usando E/S distribuída: Loop de controle de processo.

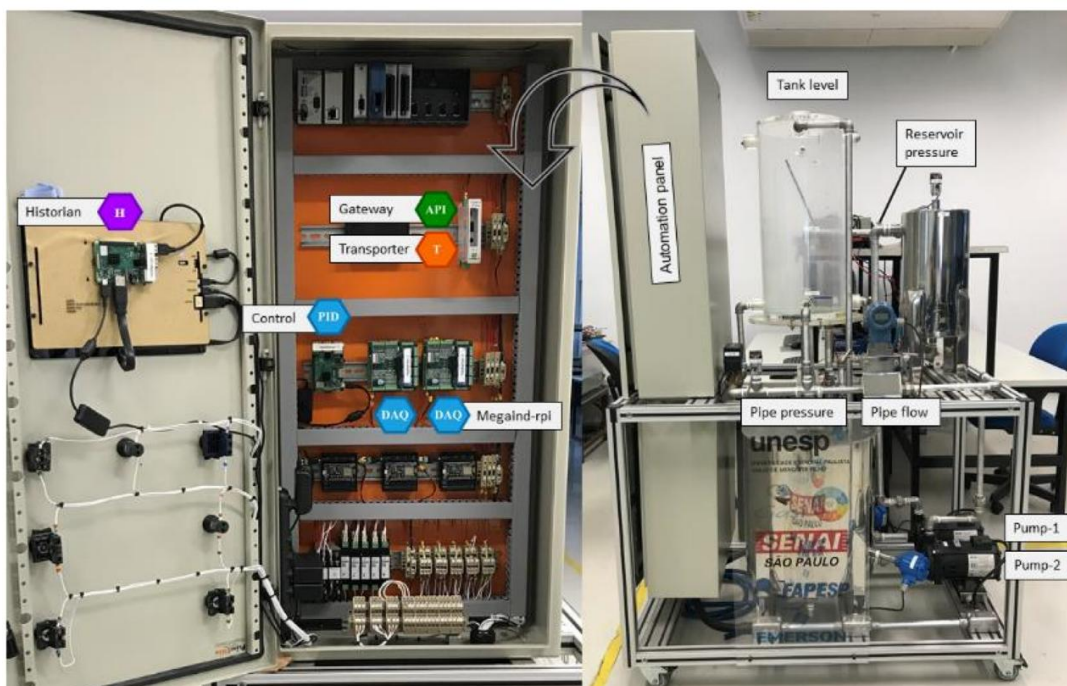


Figura 22. Planta piloto de processo industrial e as E/S distribuídas destacadas no painel elétrico.

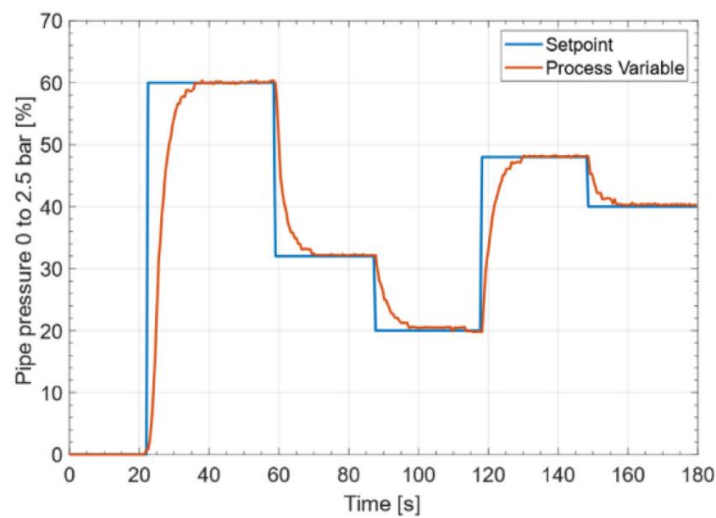


Figura 23. Exemplo de resposta de um laço de controle usando E/S distribuída.

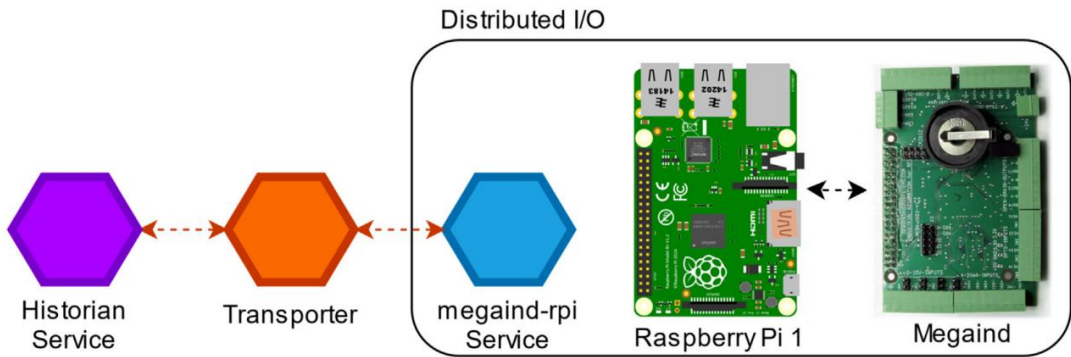


Figura 24. Exemplo de composição de aplicação Historian usando E/S distribuída.

tanque ou para o reservatório de aço inoxidável. A E/S distribuída, desenvolvida como um microserviço, é utilizada para aquisição de dados dos sensores (nível, vazão, pressão da linha e pressão do reservatório) e para comandar os atuadores (bombas com inversor de frequência e válvula proporcional). A Figura 22 mostra a E/S distribuída, serviço megaind-rpi, implantada no painel elétrico da planta piloto industrial. A Figura 23 mostra um exemplo da resposta do laço de controle de pressão da tubulação durante a operação da planta piloto industrial.

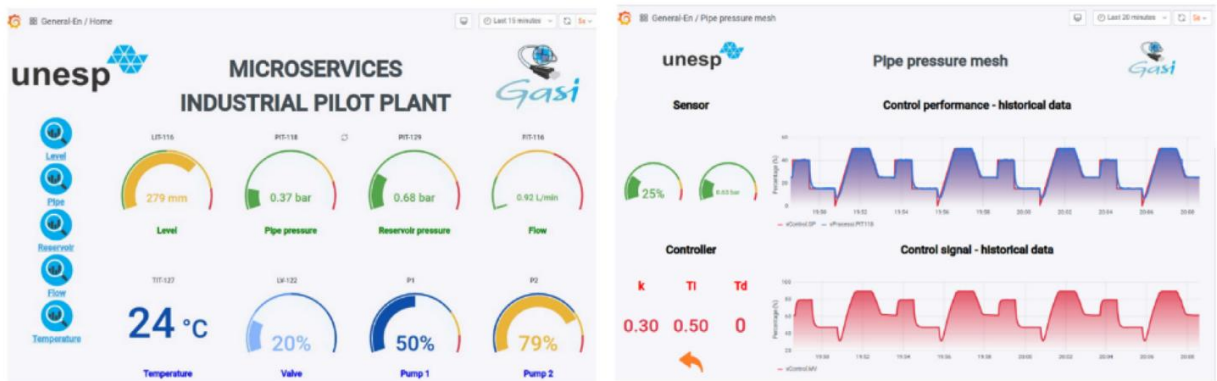


Figura 25. Painel do Grafana do serviço Process Historian.

Exemplo 4 – Historiador de Processos

O próximo exemplo também ilustra uma aplicação de pesquisa real dentro da planta piloto. Os sistemas de histórico de processos são aplicações industriais utilizadas para armazenar e analisar dados vitais de processos e da indústria. Essencialmente, são compostos por um banco de dados de séries temporais com um software IHM para visualização de dados (painel de controle).

A E/S distribuída pode ser usada para criar um aplicativo de histórico de processos. Para este exemplo, mostrado na [Figura 24](#), o serviço de histórico (hexágono roxo) solicita os dados de E/S da E/S distribuída. No serviço de histórico, os dados de E/S são solicitados ciclicamente e armazenados em um banco de dados de séries temporais (InfluxDB [25]), por exemplo, os dados de E/S são armazenados automaticamente com um registro de tempo. Carimbo (data e hora). O InfluxDB é um banco de dados de séries temporais de código aberto.

Os dados de E/S obtidos da E/S distribuída podem ser visualizados em painéis usando o Grafana [26]. O Grafana é uma solução de análise e monitoramento de código aberto com plugins de acesso a bancos de dados, simplificando os procedimentos de leitura/gravação no banco de dados para criação de interfaces de dados históricos e de HM. A [Figura 25](#) apresenta um exemplo de painel projetado para monitorar e analisar dados da planta piloto industrial em [19].

Este exemplo de aplicação destaca o potencial do uso de E/S distribuída em uma configuração de sistema histórico para IIoT e análise de dados. As informações coletadas e armazenadas pelo sistema histórico podem ser usadas futuramente para diagnóstico de processos e indicadores-chave de desempenho (KPIs). análise de desempenho, bem como ciência de dados e monitoramento de condição em aplicações relacionadas à IIoT e à Indústria 4.0 [27].

Conclusões

A aquisição de dados (DAQ) é uma etapa vital para aplicações científicas. Especificamente em aplicações industriais, a E/S distribuída é um hardware de DAQ tradicional usado para aquisição remota e em rede de dados de sensores e atuadores. O uso de arquitetura orientada a serviços (SOA) é uma alternativa promissora. As arquiteturas em aplicações de automação relacionadas à IIoT e à Indústria 4.0 são uma realidade necessária para viabilizar a interoperabilidade e a interação vertical de dispositivos e sistemas, além de permitir uma arquitetura distribuída e modular. Este artigo discute essa necessidade. Desenvolvimento e teste de uma solução de E/S distribuída, de código aberto e baixo custo, na forma de microsserviços.

A arquitetura de microsserviços aumenta a flexibilidade e a modularidade das aplicações industriais, simplificando a comunicação em rede entre serviços e aplicações. A E/S distribuída depende de soluções de código aberto para hardware (Raspberry Pi e placa de expansão de E/S industrial) e software (framework Molecular e Node.js), proporcionando uma solução confiável e acessível. principalmente quando comparado aos industriais. Seu funcionamento é semelhante ao de uma E/S distribuída tradicional, com a vantagem de ser programável, com capacidade para incorporar funcionalidades personalizadas e tornar a aquisição de dados de E/S interoperável, padronizada e escalável.

Foram apresentados diversos exemplos de aplicações para E/S distribuída, como aplicações internas e externas, e processos. Historiador e controlador de processos. As instruções de operação consideravam a montagem, instalação e utilização da E/S distribuída, juntamente com os demais serviços disponíveis na plataforma Molecular, como o API Gateway e o transportador. Estas instruções e exemplos fornecem ao leitor um primeiro passo no cenário recente de aplicações orientadas a serviços para Aplicações industriais.

O trabalho futuro se concentrará no uso de microsserviços com contêineres como o Docker para melhorar seu gerenciamento. Isso permitirá disponibilizá-los em um repositório de rede (Docker Hub) e gerenciá-los por meio de uma interface gráfica. interface (Portainer), facilitando sua implantação em comparação com o método tradicional por meio de comandos de terminal.

Declaração de contribuição de autoria CRediT

Ricardo P. Pontarolli: Curadoria de dados, Redação – rascunho original. Jeferson A. Bigheti: Visualização, Investigação. Felipe O. Domingues: Supervisão. Lucas BR de Sá: Software, Validação, Escrita – revisão e edição. Eduardo P. Godoy: Conceitualização, Metodologia, Software.

Declaração de Conflito de Interesses

Os autores declaram não possuir quaisquer interesses financeiros ou relações pessoais que possam ter influenciado o resultado do estudo. parece ter influenciado o trabalho relatado neste artigo.

Agradecimentos

Pesquisa financiada pela Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP), processo nº 2018/19984-4.

Referências

- [1] "Unipi." <https://www.unipi.tecnologia/> (Acessado em 12 de outubro de 2021).
- [2] "Módulo de expansão de E/S analógica - Raspberry Pi industrial." <https://revolutionpi.com/analog-io-module/> (Acessado em 12 de outubro de 2021).
- [3] "Iono Pi Max - PLC Industrial Raspberry Pi - E/S UPS RS232 RS485 CAN CE FCC." <https://www.sferalabs.cc/product/iono-pi-max/> (acessado em 12 de outubro, 2021).
- [4] "Monarco HAT - E/S analógica e digital, barramento RS-485 e 1-Wire para Raspberry Pi." <https://www.monarco.io/> (Acessado em 12 de outubro de 2021).

- [5] JA Basson, A. Broekman, SW Jacobsz, TD-DAQ: Um sistema de aquisição de dados de baixo custo monitorando o regime de pressão de poros não saturados em barragens de rejeitos, *HardwareX* 10 (out. 2021) e00221.
- [6] A. Broekman, S. Willem Jacobsz, H. Louw, E. Kearsley, T. Gaspar e TS da Silva Burke, "Fly-by-Pi: controle de circuito fechado de código aberto para geotécnica aplicações de teste de centrífuga", *HardwareX*, vol. 8, p. e00151, out. 2020, 10.1016/J.OHX.2020.E00151.
- [7] J. Delsing, Automação da Internet das Coisas em nuvem local: características tecnológicas e de modelo de negócios de soluções de automação da Internet das Coisas distribuídas, *IEEE Ind. Electron. Mag.* 11 (4) (dez. 2017) 8–21, <https://doi.org/10.1109/MIE.2017.2759342>.
- [8] "Qual a diferença entre E/S remota e E/S distribuída? | RealPars." <https://realpars.com/distributed-io/> (Acessado em 30 de setembro de 2021).
- [9] "Blog sobre o potencial da dupla IoT e SCADA na Indústria 4.0 | Altizon." <https://altizon.com/iot-scada-complementary-technologies/> (acessado em 30 de setembro, 2021).
- [10] Y. Wang, T. Towara e R. Anderl, "Abordagem topológica para mapeamento de tecnologias no modelo arquitetônico de referência Indústria 4.0 (RAMI 4.0)", em *Notas de aula em Engenharia e Ciência da Computação*, 2017, vol. 2.
- [11] P. Adolphs et al, *Estrutura do Shell Administrativo: Continuação do Desenvolvimento do Modelo de Referência para o Componente Indústria 4.0, ZVEI e VDI* (2016).
- [12] S. Newman, "Construindo Microserviços - Capítulo 1, 4 e 11," *Construindo Microserviços*, p. 102, 2015.
- [13] C. Richardson e F. Smith, "Projeto e implantação de microserviços com NGINX | E-book gratuito". <https://www.nginx.com/resources/library/designing-implantacao-de-microservicos> (Acessado em 07 de outubro de 2020).
- [14] M. Fowler, "Microserviços". <https://martinfowler.com/articles/microservices.html> (Acessado em 07 de outubro de 2020).
- [15] «Projeto AESOP». <https://www.imc-aesop.org/> (Acessado em 30 de setembro de 2021).
- [16] S. Karnouskos et al., "The IMC-AESOP architecture for cloud-based industrial Cyber-physical Systems," in *Industrial Cloud-Based Cyber-Physical Systems: The IMC-AESOP Approach*, vol. 9783319056241, Springer International Publishing, 2014, pp. 49–88. 10.1007/978-3-319-05624-1_3.
- [17] S. Karnouskos, T. Bangemann, C. Diedrich, Integração de dispositivos legados na futura fábrica baseada em SOA, *IFAC Proceedings Volumes* 42 (4) (jan. 2009) 2113–2118, <https://doi.org/10.3182/20090603-3-RU-2001.0487>.
- [18] JA Bigheti, MM Fernandes e E. Paciencia. Godoy, "Control as a Service: A Microservice Approach to Industry 4.0," in *2019 IEEE International Workshop on Metrology for Industry 4.0 and IoT, MetroInd 4.0 and IoT 2019 - Proceedings*, Jun. 2019, pp. 438–443. 10.1109/METROI4.2019.8792918.
- [19] RP Pontarolli, JA Bigheti, MM Fernandes, FO Domingues, SL Rizzo e EP Godoy, "Orquestração de microserviços para controle de processos na Indústria 4.0", em *2020 IEEE International Workshop on Metrology for Industry 4.0 and IoT, MetroInd 4.0 and IoT 2020 - Proceedings*, junho de 2020, pp. 245–249. 10.1109/MetroInd4.0IoT48571.2020.9138228.
- [20] M. Haizad, R. Ibrahim, A. Adnan, TD Chung e SM Hassan, "Desenvolvimento de sistema de aquisição de dados em tempo real de baixo custo para automação e controle de processos", 2º Simpósio Internacional IEEE sobre Robótica e Automação de Manufatura, ROMA 2016, fevereiro de 2017, 10.1109/ROMA.2016.7847826.
- [21] M. Abdallah e O. Elkeelany, "Um levantamento sobre sistemas de aquisição de dados (DAQ)", ICC2009 - Conferência Internacional de Computação em Engenharia e Ciência e Informação, pp. 240–243, 2009, 10.1109/ICC.2009.24.
- [22] "Moleculer - Framework progressivo de microserviços para Node.js." <https://moleculer.services/> (Acessado em 28 de janeiro de 2020).
- [23] "Placa empilhável Raspberry Pi para automação industrial." https://sequentmicrosystems.com/index.php?route=product/product&path=20&product_id=52 (Acessado em 19 de junho de 2020).
- [24] "Configuração de módulos de E/S redundantes no SIMATIC PCS 7 - ID: 28430682 - Suporte Industrial Siemens." <https://support.industry.siemens.com/cs/document/28430682/configuration-of-redundant-io-modules-in-simatic-pcs-7?dti=0&lc=en-WW> (Acessado em 18 de novembro de 2021).
- [25] "InfluxDB: Banco de dados de séries temporais de código aberto desenvolvido especificamente para esse fim | InfluxData." <https://www.influxdata.com/> (Acessado em 12 de outubro de 2021).
- [26] "Grafana: A plataforma de observabilidade aberta | Grafana Labs." <https://grafana.com/> (Acessado em 12 de outubro de 2021).
- [27] X. Gao, F. Yang, C. Shang, D. Huang, Uma revisão do monitoramento e diagnóstico do laço de controle: Perspectivas da manutenção do controlador na era do big data, *Chin. J. Química. Eng.* 24 (8) (2016) 952–962, <https://doi.org/10.1016/J.CJCHE.2016.05.039>.



Eduardo Paciencia Godoy obteve o título de Bacharel em Engenharia de Controle e Automação pela Universidade Federal de Itajubá (MG-Brasil) em 2003 e os títulos de Mestre e Doutor em Engenharia Mecânica pela Universidade de São Paulo em São Carlos (SP-Brasil) em 2007 e 2011, respectivamente. Atualmente, é Professor Associado da Universidade Estadual Paulista (Unesp) em Sorocaba (SP-Brasil). Seus interesses de pesquisa incluem redes industriais e automação, sistemas de controle em rede (NCS), redes sem fio e telemetria, eletrônica embarcada, Internet das Coisas e Indústria 4.0.



Ricardo Pasquati Pontarolli graduou-se em Engenharia de Automação Industrial pela Faculdade de Tecnologia da Informação e Comunicação (FATEC) em 2011 e obteve o título de Mestre em Engenharia Elétrica pela Universidade Estadual Paulista (Unesp) em 2020. Atualmente, é doutorando na Unesp e técnico de laboratório em automação industrial no Instituto Federal Paulista (IFSP) em Boituva. Seus interesses de pesquisa incluem automação industrial, supervisão de sistemas industriais, Internet das Coisas, eletrônica embarcada, Indústria 4.0 e mecanismos de segurança.



Jeferson André Bigheti formou-se em Engenharia Elétrica em 1990, obteve a especialização em Engenharia Mecatrônica em 2007 e os títulos de Mestre e Doutor em Engenharia Elétrica em 2011 e 2020, respectivamente, pela Universidade Estadual Paulista (Unesp) em Bauru (SP-Brasil). Atualmente, é professor do Serviço Nacional de Aprendizagem Industrial (SENAI) em Lençóis Paulista (SP-Brasil). Seus interesses de pesquisa incluem automação, supervisão de sistemas industriais, Internet das Coisas e Indústria 4.0.



Felipe de Oliveira Domingues é atualmente aluno de Engenharia de Controle e Automação na Universidade Estadual Paulista (Unesp), em Sorocaba (SP, Brasil). Seus interesses de pesquisa incluem automação industrial e Indústria 4.0.



Lucas Borges Rodrigues de Sá graduou-se em Engenharia Eletrônica pelo Instituto Nacional de Telecomunicações (Inatel) (MG-Brasil) em 2003. Atualmente, cursa mestrado em Engenharia Elétrica na Universidade Estadual Paulista (Unesp) - Sorocaba (SP). Seus interesses de pesquisa incluem automação industrial, supervisão de sistemas industriais, Internet das Coisas, eletrônica embarcada e Indústria 4.0.